

G.R.A.S.P.

Patrones Generales de Software para la Asignación de Responsabilidades

Diseño de Aplicaciones 1 · Cátedra de Ingeniería de Software

Integrante 1 · Integrante 2 · Integrante 3

Integrante 4 · Integrante 5 · Integrante 6

¿Qué es GRASP?

Principios fundamentales para **asignar responsabilidades** a objetos, expresados como **patrones**.

¿Qué es una responsabilidad? Un contrato u obligación de una clase. Por ejemplo:

- **Hacer** algo (un cálculo, crear un objeto)
- **Iniciar** una acción sobre otro objeto
- **Controlar / coordinar** actividades de otros

En diseño, las responsabilidades se traducen en:

- métodos de una clase, o
- colaboraciones entre varias clases

Un **patrón** = nombre + problema/contexto + solución.

Los 9 patrones GRASP

1 · Creador

¿quién crea?

2 · Experto

¿quién sabe?

3 · Bajo Acoplamiento

menos dependencias

4 · Alta Cohesión

responsabilidades afines

5 · Controlador

¿quién atiende eventos?

6 · Polimorfismo

variar por tipo

7 · Fabricación Pura

clase artificial

8 · Indirección

intermediario

9 · Variaciones Protegidas

interfaz estable

Objetivo transversal: **bajo acoplamiento** y **alta cohesión**.

1 · Creador

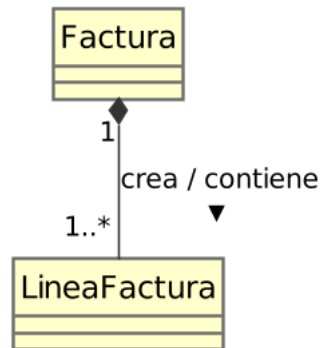
Problema: ¿quién debería crear una nueva instancia de una clase A?

Solución: asignarle la creación a la clase **B** si B...

- **agrega / contiene / registra** objetos A
- tiene los **datos de inicialización** de A
- usa A de cerca (es experta en crearlo)

Ej.: la **Factura** contiene sus **LíneaFactura** → la Factura las crea.

Creador — La Factura crea sus LineaFactura



2 · Experto en Información

Presenta: Integrante 2

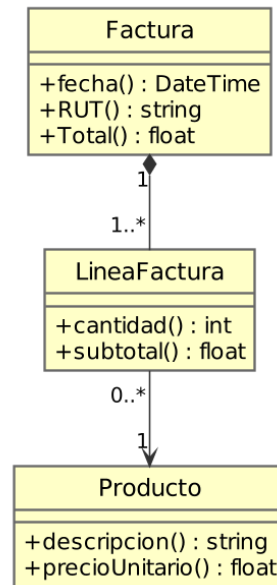
2 · Experto en Información

Problema: ¿cuál es el principio básico para asignar responsabilidades en POO?

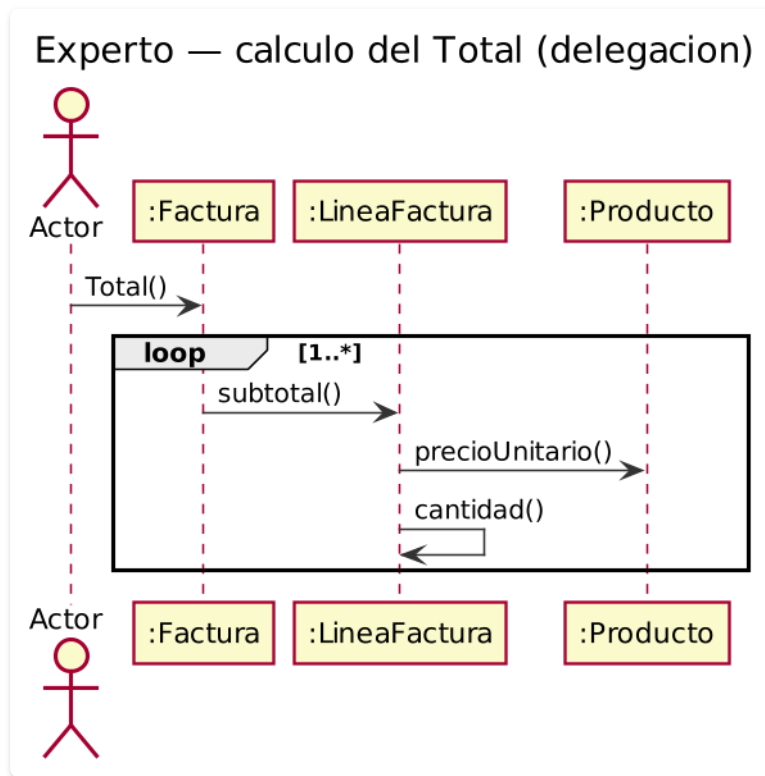
Solución: asignar la responsabilidad a la **clase que tiene la información** necesaria para cumplirla — el *experto en información*.

¿Quién conoce el **total** de la Factura? → La **Factura**: es la única que conoce todas sus líneas y sus subtotales.

Experto en Información — responsabilidades derivadas



Experto → delegación



Subtotal = `LíneaFactura.cantidad()` × `Producto.precioUnitario()` → cada quien aporta lo que sabe.

3 · Bajo Acoplamiento y Alta Cohesión

Presenta: Integrante 3

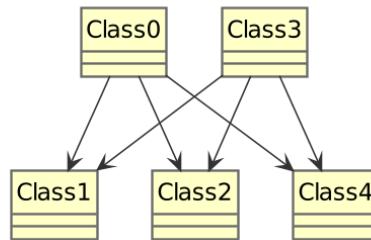
3 · Bajo Acoplamiento

Problema: ¿cómo reducir el impacto de un cambio?

Solución: asignar responsabilidades de modo que el **acoplamiento se mantenga bajo**.

- Una clase con bajo acoplamiento **depende de pocas clases**.
- Es peor aún si depende de una clase **inestable**.
- Acoplamiento 0 no existe: los objetos colaboran (es POO).

Alto acoplamiento — Class0 y Class3 dependen de muchas



Class0 y Class3: demasiadas dependencias.

3 · Alta Cohesión

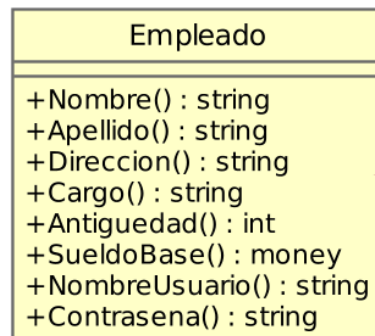
Problema: ¿cómo mantener la complejidad manejable?

Solución: asignar responsabilidades para que la clase quede **cohesiva** — que sus métodos sean *afines* y trabajen juntos.

Una clase con **baja cohesión**:

- es difícil de comprender y de reutilizar
- es delicada: los cambios la afectan seguido

Baja cohesion — metodos no afines en una clase



Mezcla datos de RRHH
con datos de seguridad
> baja cohesion

Empleado mezcla datos de RRHH con credenciales.

El yin-yang: cohesión y acoplamiento

No hay una relación matemática exacta, **pero se acompañan**:

Baja cohesión $\leftrightarrow$ normalmente **alto acoplamiento**

Una clase que "hace de todo" termina necesitando datos de muchas otras. Por eso los GRASP siempre empujan hacia **alta cohesión** y **bajo acoplamiento**.

4 · Controlador y Polimorfismo

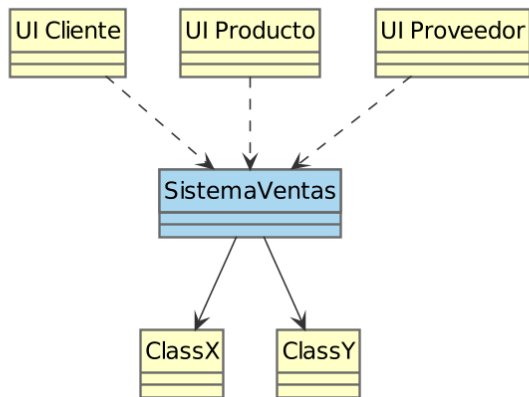
Presenta: Integrante 4

4 · Controlador

Problema: ¿quién atiende los **eventos del sistema** (acciones del usuario u otros sistemas)? **Solución:** una clase **controlador** que no sea la interfaz de usuario y que **delegue** el trabajo real.

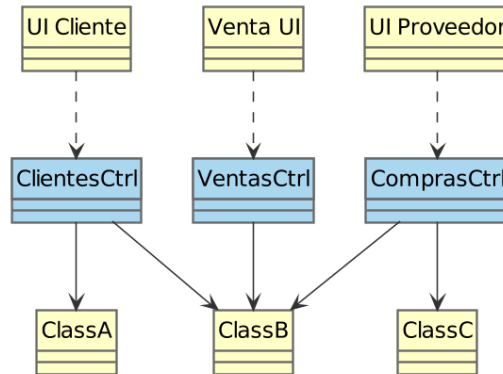
Controlador de Fachada (todo el sistema)

Controlador de Fachada — una clase para todo el sistema



Controlador de Caso de Uso (uno por caso)

Controlador de Casos de Uso — un controlador por caso de uso



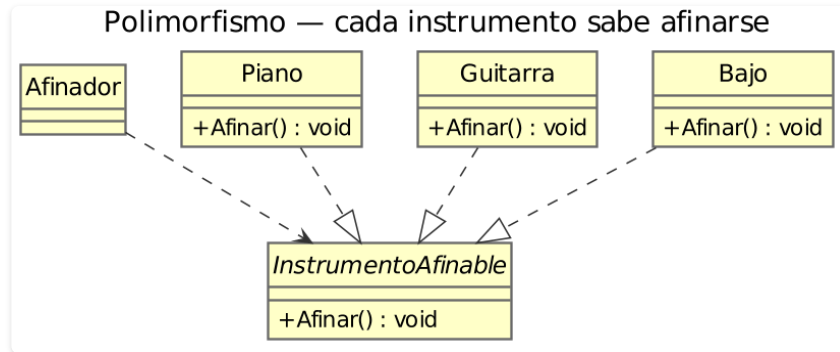
4 · Polimorfismo

Problema: ¿cómo manejar alternativas **según el tipo** de un objeto, sin llenar el código de `if` ?

Solución: distribuir el comportamiento con **operaciones polimórficas**, una por cada tipo.

Cada instrumento sabe **afinarse** a su manera → `Afinar()` en cada clase.

✓ Diseño **abierto a la extensión, cerrado al cambio** (Principio Abierto/Cerrado).



5 · Fabricación Pura e Indirección

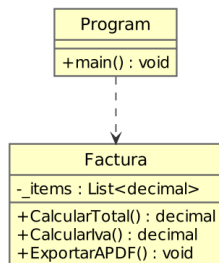
Presenta: Integrante 5

5 · Fabricación Pura

Problema: ¿a quién le doy una responsabilidad cuando ponerla en una clase del dominio **baja la cohesión o sube el acoplamiento**? **Solución:** crear una **clase artificial** (que no representa nada del dominio) con un conjunto cohesivo de responsabilidades.

Antes: Factura exporta PDF

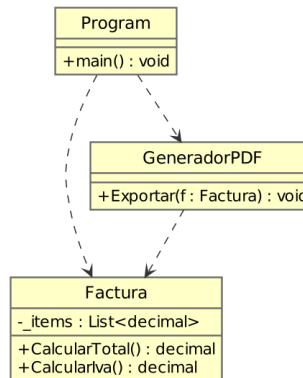
Fabricacion Pura — ANTES (Factura hace de todo)



Exportar a PDF no es
responsabilidad del dominio

Después: GeneradorPDF (fabricación pura)

Fabricacion Pura — DESPUES (clase artificial GeneradorPDF)



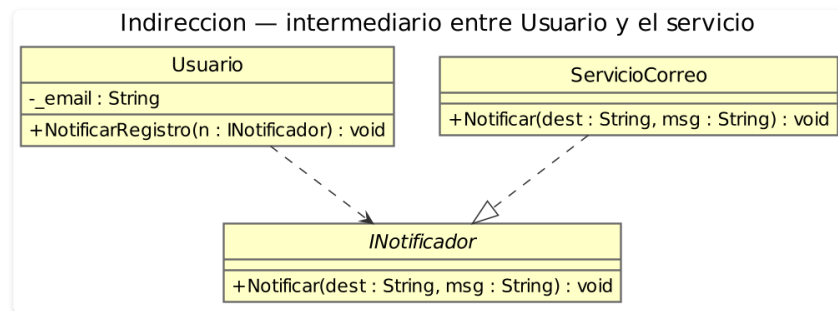
5 · Indirección

Problema: ¿cómo **desacoplar** dos elementos para que el acoplamiento sea bajo pero se mantenga el potencial de reuso?

Solución: asignar la responsabilidad a un **objeto intermediario** que medie entre ambos.

El **Usuario** no conoce al **ServicioCorreo** concreto: habla con la interfaz **INotificador**.

Un intermediario de indirección suele ser también una **Fabricación Pura**.



6 · Variaciones Protegidas

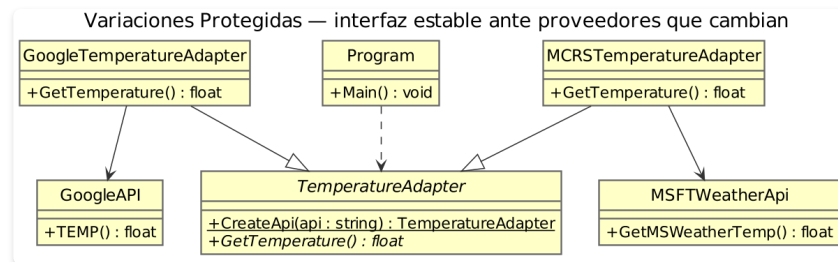
Presenta: Integrante 6

6 · Variaciones Protegidas

Problema: ¿cómo diseñar para que las **variaciones** de una parte no impacten al resto?

Solución: identificar los puntos que pueden variar y **envolverlos con una interfaz estable**.

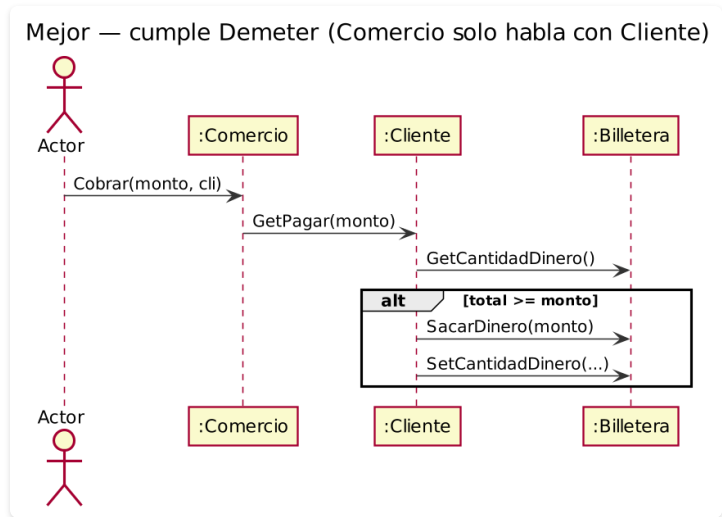
- no acoplarse a tipos concretos
- favorecer el polimorfismo
- desacoplar con indirección



Cambiar de proveedor solo afecta a su adaptador.

Ley de Demeter

"No hables con extraños": una clase debe evitar conocer la **estructura interna** de objetos indirectos.



✗ El Comercio pide la Billetera del Cliente y opera sobre ella → queda acoplado a la estructura interna.

✓ El Comercio le pide al **Cliente** que pague (`GetPagar`) → el Cliente maneja su Billetera. **Menos acoplamiento.**

En resumen

Aplicar cualquier GRASP **nunca** debe ir en contra de las métricas del buen diseño: **alta cohesión** y **bajo acoplamiento**.

Creador · Experto

Bajo Acoplamiento · Alta Cohesión

Controlador

Polimorfismo

Fabricación Pura · Indirección

Variaciones Protegidas

¡Gracias! ¿Preguntas?